Title: Azure SRE Agent GA 後の整理ノート（権限・実行モード・拡張・PoC 評価）
Date: 2026-03-13
Modified: 2026-04-03
Slug: sre-agent-planning
Lang: ja-jp
Category: notebook
Tags: azure, sre-agent, aiops, observability, incident-management
Summary: Azure SRE Agent GA 後の Microsoft Learn ドキュメントを章別に再精査し、アクセス制御・実行モード・拡張機能・PoC 評価フレームワークを整理したノート。

本稿は、2026-04-03 時点の Microsoft Learn に公開されている Azure SRE Agent 関連ドキュメントを章単位で再精査し、GA 後の事実だけに絞って再構成したノートです。

> 初版 (2026-03-13) からの主な更新:
>
> - 2026-03-28 付で更新された docs に合わせ、アクセス制御を「4レイヤー」から「**3レイヤー**」に修正（OBO はエージェント権限レイヤーの一部として整理）。
> - Run modes の既定値を最新 docs に合わせて修正（response plan・scheduled task ともに既定 Autonomous、エージェントレベル既定は Review）。
> - 全章のリファレンス URL を再検証。

引用の出し方:

- 本文中は [章-番号] の参照番号のみを付けます。
- 各章の末尾に、対応する Microsoft Learn の URL と短い引用をまとめます。

Architecture

アジェンダ

1. SRE Agent は何をするサービスか
2. 導入前提とロール設計
3. アクセス制御の 3 レイヤー
4. エージェント権限と OBO
5. Run modes（Review / Autonomous）
6. Scheduled tasks
7. Incident platforms と response plans
8. Memory system
9. Subagents / Connectors / Python tools / MCP
10. FAQ から見た GA 後の注意点
11. GA 後に読み替えるべきポイント
12. まとめ
13. PoC 評価フレームワーク

---

0. 参照した公式ページ

- Overview[0-1]
- Create and use an agent[0-2]

- User roles and permissions[0-3]
- Agent permissions[0-4]
- Run modes[0-5]
- Scheduled tasks[0-6]
- Incident platforms[0-7]
- Incident response plans[0-8]
- Memory and knowledge[0-9]
- Subagents[0-10]
- Connectors[0-11]
- Python tools[0-12]
- MCP connector tutorial[0-13]
- FAQ[0-14]

参考（第0章）

- [0-1] https://learn.microsoft.com/ja-jp/azure/sre-agent/overview — 「運用作業を自動化し、作業の手間を軽減」
- [0-2] https://learn.microsoft.com/ja-jp/azure/sre-agent/usage — 「`Microsoft.Authorization/roleAssignments/write`」
- [0-3] https://learn.microsoft.com/ja-jp/azure/sre-agent/user-roles — 「アクセス制御は、3 つのレイヤーで機能」
- [0-4] https://learn.microsoft.com/ja-jp/azure/sre-agent/agent-managed-identity — 「すべてのエージェントにはマネージド ID があります」
- [0-5] https://learn.microsoft.com/ja-jp/azure/sre-agent/agent-run-modes — 「トリガーごと、およびスケジュールされたタスクごとに」
- [0-6] https://learn.microsoft.com/ja-jp/azure/sre-agent/scheduled-tasks — 「監視、メンテナンス、セキュリティ チェックなどのワークフローを自動化」
- [0-7] https://learn.microsoft.com/ja-jp/azure/sre-agent/incident-management — 「Azure Monitor は既定で接続されています」
- [0-8] https://learn.microsoft.com/ja-jp/azure/sre-agent/incident-response-plan — 「インシデント フィルターをサブエージェントに接続」
- [0-9] https://learn.microsoft.com/ja-jp/azure/sre-agent/memory-system — 「過去のインシデントで何が機能していたかを思い出し」
- [0-10] https://learn.microsoft.com/ja-jp/azure/sre-agent/sub-agents — 「オンデマンドで呼び出すスペシャリスト エージェント」
- [0-11] https://learn.microsoft.com/ja-jp/azure/sre-agent/connectors — 「外部システムにまで拡張」
- [0-12] https://learn.microsoft.com/ja-jp/azure/sre-agent/custom-logic-python — 「内部 API、オンプレミス データベース、マルチクラウド」
- [0-13] https://learn.microsoft.com/ja-jp/azure/sre-agent/custom-mcp-server — 「MCP コネクタをエージェントに追加する」
- [0-14] https://learn.microsoft.com/ja-jp/azure/sre-agent/faq — 「一般的な FAQ」

---

1. SRE Agent は何をするサービスか

説明

Azure SRE Agent は、運用作業を自動化し、SRE や運用チームの手作業を減らすためのサービスです。[1-1]

GA 後の overview では、単なる「監視の AI 助手」ではなく、Azure サービスと外部システムをつなぎ、運用ワークフローを end-to-end で自動化するプラットフォームとして説明されています。[1-2]

また、Azure CLI と REST API を通じて、Azure の各種サービスを横断的に管理できる前提が明記されています。[1-3]

スライド要点

- 監視・トラブルシュート・是正だけでなく、運用ワークフロー全体の自動化が主語になっている。[1-2]
- Azure ネイティブの知識に加えて、カスタム runbook、subagent、外部統合で拡張する設計である。[1-4]
- チャット UI は引き続き英語のみで、社内展開時はプロンプトや runbook も英語前提で考える必要がある。[1-5]
- エージェント作成時には Application Insights、Log Analytics workspace、Managed Identity が自動作成される。[1-6]

参考（第1章）

- [1-1] https://learn.microsoft.com/ja-jp/azure/sre-agent/overview — 「運用作業を自動化し、作業の手間を軽減」
- [1-2] https://learn.microsoft.com/ja-jp/azure/sre-agent/overview — 「システムを接続し、ワークフローをエンドツーエンドで自動化するための AI 駆動型プラットフォーム」
- [1-3] https://learn.microsoft.com/ja-jp/azure/sre-agent/overview — 「Azure CLI と REST API を使用して、すべての Azure サービスを管理できます」
- [1-4] https://learn.microsoft.com/ja-jp/azure/sre-agent/overview#how-does-sre-agent-work — 「Built-in Azure knowledge / Custom runbooks / Subagent extensibility / External integrations」
- [1-5] https://learn.microsoft.com/ja-jp/azure/sre-agent/overview#get-started — 「チャット インターフェイスでサポートされている唯一の言語は英語です」
- [1-6] https://learn.microsoft.com/ja-jp/azure/sre-agent/overview#get-started — 「Azure Application Insights / Log Analytics workspace / Managed Identity」

---

2. 導入前提と初期セットアップ

説明

エージェント作成時の前提は、現行 docs でも次の 3 点が中心です。[2-1][2-2][2-3]

- `Microsoft.Authorization/roleAssignments/write` を持つこと
- `*.azuresre.ai` への到達性があること
- サポート対象リージョンに作成できること

FAQ では利用可能リージョンとして、スウェーデン中部、米国東部 2、オーストラリア東部が挙げられています。[2-4]

スライド要点

- 最初に詰まりやすいのは「Azure RBAC」と「ファイアウォール」の 2 点。[2-1][2-2]
- usage ではエージェント作成リージョンとして米国東部 2 が案内されているが、FAQ 上は提供リージョンが複数ある。[2-3][2-4]

- エージェント自体の配置リージョンと、調査対象リソースのリージョンは別に考えてよい。FAQ では、デプロイ後は他リージョンの Azure リソースも管理・調査できると説明されている。[2-5]

参考（第2章）

- [2-1] https://learn.microsoft.com/ja-jp/azure/sre-agent/usage — 「`Microsoft.Authorization/roleAssignments/write` または ユーザー アクセス管理者」
- [2-2] https://learn.microsoft.com/ja-jp/azure/sre-agent/usage — 「`*.azuresre.ai` を追加します」
- [2-3] https://learn.microsoft.com/ja-jp/azure/sre-agent/usage — 「リージョン: 米国東部 2 を選択します」
- [2-4] https://learn.microsoft.com/ja-jp/azure/sre-agent/faq — 「スウェーデン中部 / 米国東部 2 / オーストラリア東部」
- [2-5] https://learn.microsoft.com/ja-jp/azure/sre-agent/faq — 「エージェント自体がホストされている場所に関係なく、すべての Azure リージョンのリソースを管理および調査できます」

顧客向けにそのまま使える導入整理

お客さん向けに一言で整理するなら、まずは「SRE Agent をデプロイする担当者は、少なくとも SRE Agent を管理させるスコープで Owner 相当が必要」と説明するのが実務上わかりやすいです。[2-6][2-7][2-8]

理由は、初期導入では次の 2 種類の権限が同時に要るためです。[1-6][2-6][2-7][2-8]

- エージェント用の関連リソースを作る権限
- エージェントのマネージド ID に RBAC を割り当てる権限

前者だけなら共同作成者でも足りる場面がありますが、後者は `Microsoft.Authorization/roleAssignments/write` が要るため、User Access Administrator だけでも、逆に Contributor だけでも片手落ちになりやすいです。[2-1][2-6][2-8]

そのため、導入初期の説明としては次の形が安全です。

観点	顧客向けの説明
デプロイ担当者	SRE Agent を作成し、必要な RBAC を付与できる人
推奨ロール	対象スコープの Owner が最も説明しやすい
User Access Administrator の位置づけ	RBAC 割り当てには有効だが、リソース作成権限は別途必要
管理スコープ	まずは対象リソース グループを限定して始める

特に会話上は、「SRE Agent が管理するスコープの Owner」という表現で大きくは間違っていません。ただし厳密には、エージェントを配置する側のスコープでもリソース作成権限が必要です。したがって、実運用では「配置先スコープと管理対象スコープの両方で、必要な作成権限と RBAC 付与権限を持つ担当者」と補うのが正確です。[1-6][2-6][2-8]

顧客説明用の導入ステップ

お客さん向けの進め方としては、次の 5 段階で整理すると通しやすいです。[2-2][2-4][2-6]

1. SRE Agent を配置するリージョンとリソース グループを決める。
2. ネットワーク要件として `*.azuresre.ai` への疎通を確認する。
3. デプロイ担当者が、必要な Azure リソース作成権限と RBAC 付与権限を持っていることを確認する。

4. 最初は限定したリソース グループだけを SRE Agent の管理対象にする。
5. 最初は Reader 系で診断中心に始め、変更操作が必要になった範囲だけ write 権限や OBO を追加する。

この言い方にしておくと、「最初から広い権限で全面導入するものではなく、限定スコープで安全に始めるサービス」という位置づけが伝わりやすいです。[2-6][4-3][4-5]

- [2-6] https://learn.microsoft.com/ja-jp/azure/sre-agent/create-agent — 「前提条件 / 所有者またはユーザー アクセス管理者」
- [2-7] https://learn.microsoft.com/ja-jp/azure/role-based-access-control/built-in-roles#owner — 「Owner」
- [2-8] https://learn.microsoft.com/ja-jp/azure/role-based-access-control/built-in-roles#user-access-administrator — 「User Access Administrator」

---

3. アクセス制御は「3レイヤー」で整理する

説明

最新の docs（2026-03-28 更新）では、アクセス制御は **3 レイヤー**で説明されています。[3-1]

レイヤー	何を制御するか	設定場所
ユーザー ロール	エージェント上でユーザーが何をできるか	エージェントリソースの Azure IAM
実行モード	エージェントが実行前に承認を求めるか	response plan / scheduled task ごと
エージェント権限	エージェントが Azure 上で何にアクセスできるか（OBO フォールバック含む）	リソースグループの RBAC ロール

> **初版からの変更**: 旧稿では OBO を独立した第4レイヤーとしていましたが、最新 docs ではエージェント権限レイヤーの一部（Managed Identity RBAC + OBO fallback）として統合されています。[3-1][3-2]

組み込みロール

エージェントの組み込みロールは 3 種類です。[3-3]

ロール	できること	できないこと
SRE Agent Reader	スレッド、ログ、インシデントの閲覧	チャット、アクション要求、設定変更
SRE Agent Standard User	チャット、診断、アクション要求	アクション承認、コネクタ管理、削除
SRE Agent Administrator	アクション承認、コネクタ管理、設定変更、削除	—

エージェント作成者には自動で Administrator が付与されます。[3-4]

ポータルの権限強制

ポータルは 2 段階で権限を強制します。[3-5]

1. **レベル1**: ロール未割り当てなら Access Required 画面を表示
2. **レベル2**: ロールがあっても権限外の操作はバックエンドが 403 で拒否

> UI 上でボタンが表示される場合があっても、バックエンドは正しく止めると明記されています。ただし、この UI 側の無効化はまだ全機能で一貫していないとの注記もあります。[3-5]

スライド要点

- 「誰が押せるか」と「エージェントが何を実行できるか」は別問題として分けて説明する。[3-1]
- Standard User は診断や提案まで、Administrator は承認と構成変更まで担当する設計が基本線。[3-3]
- ロール割り当ては Azure CLI でも可能。`az role assignment create --role "SRE Agent Standard User"` [3-6]

参考（第3章）

- [3-1] https://learn.microsoft.com/en-us/azure/sre-agent/user-roles — 「Access control works across three layers」
- [3-2] https://learn.microsoft.com/en-us/azure/sre-agent/user-roles — 「User roles / Run modes / Agent permissions」
- [3-3] https://learn.microsoft.com/en-us/azure/sre-agent/user-roles — 「SRE Agent Reader / Standard User / Administrator」
- [3-4] https://learn.microsoft.com/en-us/azure/sre-agent/user-roles — 「The user who creates the agent automatically gets the SRE Agent Administrator role」
- [3-5] https://learn.microsoft.com/en-us/azure/sre-agent/user-roles — 「the backend blocks the action with a 403 error」「this behavior isn't yet consistent across all features」
- [3-6] https://learn.microsoft.com/en-us/azure/sre-agent/user-roles — 「az role assignment create」

---

4. エージェント権限は RBAC 割り当てで考える

説明

現行の managed identity ドキュメントは、旧稿のような「Reader / Privileged の二択」というより、マネージド ID に対する Azure RBAC 割り当てと OBO フォールバックで整理されています。[4-1][4-2]

リソース グループをエージェントに割り当てると、最低限の診断用ロールとして Reader、Log Analytics Reader、Monitoring Reader が自動で付与されます。[4-3]

書き込み系をさせるには、共同作成者や必要なアクション ロールを追加で付与します。[4-4]

OBO の位置づけ

エージェントのマネージド ID に必要権限がない場合、管理者が OBO を承認して、一時的にユーザー権限を使わせることができます。[4-5]

この一時権限は保持されず、操作完了後にエージェントはマネージド ID ベースへ戻ります。[4-6]

スライド要点

- まず Reader 系ロールで診断基盤を作り、必要な write を個別に足すのが現行 docs に近い。[4-3][4-4]
- OBO は「常用権限」ではなく、権限不足時の一時承認である。[4-5][4-6]
- アクセスを減らすときは個別権限の差し引きではなく、スコープからリソース グループを外す。[4-7]

参考（第4章）

- [4-1] https://learn.microsoft.com/ja-jp/azure/sre-agent/agent-managed-identity — 「すべてのエージェントにはマネージド ID があります」
- [4-2] https://learn.microsoft.com/ja-jp/azure/sre-agent/agent-managed-identity — 「リソース グループをエージェントに割り当て、マネージド ID に RBAC ロールを付与」
- [4-3] https://learn.microsoft.com/ja-jp/azure/sre-agent/agent-managed-identity — 「Reader / Log Analytics 閲覧者 / 監視リーダー」
- [4-4] https://learn.microsoft.com/ja-jp/azure/sre-agent/agent-managed-identity — 「共同作成者または特定のアクション ロールを付与」
- [4-5] https://learn.microsoft.com/ja-jp/azure/sre-agent/agent-managed-identity — 「オンビハーフ・オブ (OBO)」
- [4-6] https://learn.microsoft.com/ja-jp/azure/sre-agent/agent-managed-identity — 「アクセス許可は保持されません」
- [4-7] https://learn.microsoft.com/ja-jp/azure/sre-agent/agent-managed-identity — 「個々のアクセス許可を削除することはできません。リソース グループ全体のみ」

---

5. Run modes（Review / Autonomous）

説明

Run modes は、エージェントが実行前に承認を求めるかどうかを制御します。権限（Permissions）はリソースへのアクセス可否を制御するもので、Run modes とは別軸です。両方の条件が揃って初めてアクションが実行されます。[5-1]

2 つのモード

モード	動作	推奨用途
Review	エージェントがアクションを提案し、ユーザーが承認/拒否	本番システム、重要インフラ
Autonomous	エージェントが即座に実行し、事後報告	非本番環境、信頼できる繰り返しタスク

> **重要**: Review mode の承認ボタンが表示されるのは Azure インフラ操作（Azure CLI コマンド、ARM 操作等の書き込みアクション）のみです。メール送信、Teams 投稿、外部データソースへのクエリ等は、利用可能なツールと権限に基づいて進行します。[5-2]

設定場所

実行モードは **response plan ごと**および **scheduled task ごと**に設定します。エージェントレベルでは直接設定しません。[5-3]

対象	既定モード	選択可能
Incident response plan	Autonomous	Review, Autonomous
Scheduled task	Autonomous	Review, Autonomous

エージェントの Settings > Basics にはグローバルモード（既定 Review）がありますが、これは response plan / task で個別設定がない場合のフォールバックです。[5-4]

> **初版からの変更**: 旧稿ではグローバル既定が Review で scheduled task 既定も Review と記載していましたが、最新 docs では response plan・scheduled task ともに既定は **Autonomous** です。[5-3]

読み取り/書き込みと権限の組み合わせ

読み取りアクション:

エージェント権限あり	モード	動作
あり	Review / Autonomous	そのまま実行
なし	Review / Autonomous	OBO で一時アクセスを要求

書き込みアクション:

エージェント権限あり	モード	動作
あり	Review	同意を求めてから実行
なし	Review	同意を求め、さらに OBO を要求
あり	Autonomous	そのまま実行
なし	Autonomous	OBO で一時アクセスを要求

[5-5][5-6]

docs 推奨の運用ガイダンス

シナリオ	推奨モード
本番インシデント	Review
ステージング/開発インシデント	Autonomous
日次ヘルスチェック	Autonomous
コスト・使用量レポート	Autonomous
セキュリティアラート	Review

Review mode で 2〜4 週間運用し、繰り返し承認しているパターンだけを Autonomous に切り替えるのが推奨されています。[5-7]

スライド要点

- Autonomous の可否は「incident かどうか」ではなく、「response plan / task 設定」と「エージェントレベルのフォールバック」で決まる。[5-3][5-4]
- Autonomous でも権限不足は消えない。承認フローと RBAC は別軸で残る。[5-5][5-6]
- まずは Review で始め、パターンが安定してから段階的に Autonomous 化する。[5-7]

参考（第5章）

- [5-1] https://learn.microsoft.com/en-us/azure/sre-agent/run-modes — 「Run modes control the approval workflow … Permissions control resource access … The agent needs both conditions satisfied to act」
- [5-2] https://learn.microsoft.com/en-us/azure/sre-agent/run-modes — 「Review mode shows Approve and Deny buttons only for Azure infrastructure operations」
- [5-3] https://learn.microsoft.com/en-us/azure/sre-agent/run-modes — 「Set run modes per response plan and per scheduled task」「Default: Autonomous」
- [5-4] https://learn.microsoft.com/en-us/azure/sre-agent/run-modes — 「Settings > Basics shows the agent's global mode … defaults to Review … serves as the fallback」
- [5-5] https://learn.microsoft.com/en-us/azure/sre-agent/run-modes — 「Read-only actions」
- [5-6] https://learn.microsoft.com/en-us/azure/sre-agent/run-modes — 「Write actions」
- [5-7] https://learn.microsoft.com/en-us/azure/sre-agent/run-modes — 「Start with review mode. Observe what the agent recommends for two to four weeks」

---

6. Scheduled tasks は proactive 運用の主役

説明

scheduled tasks は、監視、メンテナンス、セキュリティチェックなどのワークフローを定義したスケジュールで実行する機能です。[6-1]

現行 docs では、手動作成、チャット中の要求、incident response の一部としての自動生成の 3 経路が明示されています。[6-2]

スライド要点

- 事故が起きてから動く reactive だけでなく、定期ヘルスチェックやセキュリティ確認を proactive に自動化するのが scheduled tasks の役割。[6-1]
- スケジュール記述は英語で入力し、`Draft the cron` で cron 式に落とす補助が使える。[6-3]
- 指示文は `Polish instructions` で改善できる。[6-4]

参考（第6章）

- [6-1] https://learn.microsoft.com/ja-jp/azure/sre-agent/scheduled-tasks — 「監視、メンテナンス、セキュリティ チェックなどのワークフローを自動化」
- [6-2] https://learn.microsoft.com/ja-jp/azure/sre-agent/scheduled-tasks — 「手動で作成 / チャット中に要求 / インシデント対応の一部として自律的に生成」
- [6-3] https://learn.microsoft.com/ja-jp/azure/sre-agent/scheduled-tasks — 「Cron の下書き」
- [6-4] https://learn.microsoft.com/ja-jp/azure/sre-agent/scheduled-tasks — 「ポーランド語の指示」

---

7. Incident platforms と response plans は「サブエージェントへのルーティング」で理解する

Incident platforms

incident platform は、問題が起きたときにエージェントへ通知するシステムです。[7-1]

サポート対象は Azure Monitor、PagerDuty、ServiceNow です。[7-2]

Azure Monitor は既定で接続されますが、一度に有効にできる incident platform は 1 つだけです。別のプラットフォームへ切り替えると Azure Monitor は切断されます。[7-3]

Response plans

GA 後の docs で重要なのは、response plan が「フィルター + サブエージェント ハンドラー」の組み合わせとして説明されている点です。[7-4]

つまり、response plan は単なる実行モード設定ではなく、「どのインシデントを、どの専門サブエージェントへ流すか」を定義するルーティングでもあります。[7-4][7-5]

要素	制御する内容
Incident filters	どのインシデントを拾うか
Response subagent	どの専門家に処理させるか
Autonomy level	Review / Autonomous

既定値と quickstart の読み分け

ここは docs が 2 系統あります。[7-6][7-7]

- incident-response-plan の default settings: Azure Monitor 接続、低優先度、Review mode
- incident-management の quickstart response plan: Azure Monitor なら Sev3、PagerDuty なら P1、Autonomous、`quickstart_handler`

矛盾ではなく、「一般的な既定プラン」と「incident platform 接続時のクイックスタート自動生成」を分けて読んだ方が理解しやすいです。[7-6][7-7]

テスト

response plan は過去インシデントでテストでき、test mode は常に read-only です。[7-8]

スライド要点

- incident platform はトリガー、connectors は調査に使う外部データ・アクション、という役割分担で説明する。[7-9]
- response plan は「条件分岐」ではなく「サブエージェントへの自動ルーティング」と捉えると整理しやすい。[7-4][7-5]
- quickstart plan と通常の default settings は別物として説明した方が誤読を防げる。[7-6][7-7]

参考（第7章）

- [7-1] https://learn.microsoft.com/ja-jp/azure/sre-agent/incident-management — 「問題が発生したときにエージェントに通知するシステム」
- [7-2] https://learn.microsoft.com/ja-jp/azure/sre-agent/incident-management — 「Azure Monitor / PagerDuty / ServiceNow」
- [7-3] https://learn.microsoft.com/ja-jp/azure/sre-agent/incident-management — 「一度にアクティブにできるインシデント プラットフォームは 1 つだけ」
- [7-4] https://learn.microsoft.com/ja-jp/azure/sre-agent/incident-response-plan — 「インシデント フィルターをサブエージェントに接続します」

- [7-5] https://learn.microsoft.com/ja-jp/azure/sre-agent/incident-response-plan — 「response subagent / autonomy level」
- [7-6] https://learn.microsoft.com/ja-jp/azure/sre-agent/incident-response-plan#create-a-response-plan — 「低優先度インシデント / review mode」
- [7-7] https://learn.microsoft.com/ja-jp/azure/sre-agent/incident-management — 「quickstart_handler / Azure Monitor: Sev3 / PagerDuty: P1 / 完全自律モード」
- [7-8] https://learn.microsoft.com/ja-jp/azure/sre-agent/incident-response-plan — 「テスト モードの場合、エージェントは常に読み取り専用モード」
- [7-9] https://learn.microsoft.com/ja-jp/azure/sre-agent/incident-management — 「インシデント プラットフォーム vs. コネクタ」

---

8. Memory system は「会話の学習」と「運用知識の供給」を分けて考える

説明

memory system の中心は、過去のインシデント、ユーザーメモリ、ナレッジベースを横断検索し、根拠付きで答えることです。[8-1]

さらに docs では、会話後 30 分程度で自動生成される session insights と、`memories/synthesizedKnowledge/` 以下へ永続化される knowledge files も説明されています。[8-2][8-3]

実運用で押さえる点

- `#remember` / `#retrieve` / `#forget` で個別ファクトを扱う。[8-4]
- アップロード文書は `.md` / `.txt` が基本。[8-5]
- 秘密情報、資格情報、API キーは保存しない。[8-6]
- session insights は、症状、解決手順、根本原因、落とし穴を自動抽出する。[8-7]

ドキュメント間の注意点

knowledge base のファイル上限は、memory-system ページでは 16 MB/ファイル、sub-agents ページでは 50 MB/ファイルと記述が分かれています。[8-5][8-8]

本稿では、agent 全体の memory/knowledge の説明としては memory-system ページの 16 MB を優先し、subagent 側の knowledge base 管理では別表記があることを注記しておきます。[8-5][8-8]

スライド要点

- まず `#remember` と knowledge base で最低限の運用前提を入れ、session insights で抜け漏れを埋める導入順が docs に沿っている。[8-4][8-7]
- memory は「テレメトリ保存先」ではなく、手順・前提・学習の保存先である。[8-1]
- ファイル上限の記述にはページ差異があるため、運用前に portal 実挙動を確認した方がよい。[8-5][8-8]

参考（第8章）

- [8-1] https://learn.microsoft.com/ja-jp/azure/sre-agent/memory-system — 「過去のインシデント / ユーザーメモリ / ナレッジ ベース」
- [8-2] https://learn.microsoft.com/ja-jp/azure/sre-agent/memory-system — 「セッション分析情報」

- [8-3] https://learn.microsoft.com/ja-jp/azure/sre-agent/memory-system — 「`memories/synthesizedKnowledge/`」
- [8-4] https://learn.microsoft.com/ja-jp/azure/sre-agent/memory-system — 「`#remember` / `#retrieve` / `#forget`」
- [8-5] https://learn.microsoft.com/ja-jp/azure/sre-agent/memory-system — 「`.md` or `.txt` files (up to 16 MB each)」
- [8-6] https://learn.microsoft.com/ja-jp/azure/sre-agent/memory-system — 「Don't store secrets, credentials, API keys」
- [8-7] https://learn.microsoft.com/ja-jp/azure/sre-agent/memory-system — 「症状、解決手順、根本原因、および落とし穴」
- [8-8] https://learn.microsoft.com/ja-jp/azure/sre-agent/sub-agents — 「ファイルあたり最大 50 MB」

---

9. Subagents / Connectors / Python tools / MCP

Subagents

subagent は、`/agent` で明示的に呼び出すスペシャリスト エージェントです。[9-1]

skills が常時利用可能なのに対し、subagent は明示呼び出しまたはトリガー接続で使う点が違います。[9-2]

また、subagent builder には canvas view、table view、test playground があり、トリガーやツールの接続を視覚的に確認できます。[9-3]

Connectors

connectors は、Azure 外部のシステムへエージェントを拡張するための仕組みです。[9-4]

docs 上は次の 4 カテゴリに整理できます。[9-5]

- data sources
- source code and knowledge
- collaboration tools
- custom connectors (MCP servers)

一方で Azure Monitor、Application Insights、Log Analytics、Azure Resource Graph、ARM/Azure CLI、AKS diagnostics などは、コネクタなしでも built-in capability として使えます。[9-6]

Python tools

Python tools は、内部 API、オンプレ DB、マルチクラウド、独自ロジックをエージェントへ追加する拡張方法です。[9-7]

実行環境は 5〜900 秒の timeout、毎回新しいコンテナー、`/mnt/data` の一時ファイル、送信ネットワーク有効、JSON シリアル化可能な戻り値、という制約で整理されています。[9-8]

MCP

MCP connector は external tool / service への接続方法として docs がかなり整理されました。[9-9]

現在の docs で押さえる点は次のとおりです。

- Builder > Connectors から MCP サーバーを追加する。[9-10]

- 接続状態は Connected / Disconnected / Failed / Initializing / Unavailable で監視できる。[9-11]
- subagent へは個別ツール追加も、`{connection-id}/*` のワイルドカード追加もできる。[9-12]
- playground で接続済み subagent をテストできる。[9-13]

スライド要点

- Azure 内部の調査は built-in tools でかなり完結する。connectors は Azure 外部へ伸ばすときの仕組み。[9-4][9-6]
- subagent は専門知識とツール束をパッケージ化し、response plan や scheduled task から呼ばれる単位として理解するとよい。[9-1][9-3]
- MCP は connector 側の正常性監視と wildcard 追加が docs で明文化され、運用設計しやすくなった。[9-11][9-12]

参考（第9章）

- [9-1] https://learn.microsoft.com/ja-jp/azure/sre-agent/sub-agents — 「オンデマンドで呼び出すスペシャリスト エージェント」
- [9-2] https://learn.microsoft.com/ja-jp/azure/sre-agent/sub-agents — 「スキルとは異なり、サブエージェントには明示的な呼び出しが必要」
- [9-3] https://learn.microsoft.com/ja-jp/azure/sre-agent/sub-agents — 「キャンバス ビュー / テーブル ビュー / テストプレイグラウンド」
- [9-4] https://learn.microsoft.com/ja-jp/azure/sre-agent/connectors — 「外部システム (Kusto クラスター、ソース コード リポジトリ、コラボレーション ツール、カスタム API) にまで拡張」
- [9-5] https://learn.microsoft.com/ja-jp/azure/sre-agent/connectors — 「4 つのカテゴリ」
- [9-6] https://learn.microsoft.com/ja-jp/azure/sre-agent/connectors — 「コネクタなしでエージェントができること」
- [9-7] https://learn.microsoft.com/ja-jp/azure/sre-agent/custom-logic-python — 「内部 API、オンプレミス データベース、マルチクラウド プラットフォーム」
- [9-8] https://learn.microsoft.com/ja-jp/azure/sre-agent/custom-logic-python — 「タイムアウト 5 ~ 900 秒 / `/mnt/data` / JSON 出力が必要」
- [9-9] https://learn.microsoft.com/ja-jp/azure/sre-agent/custom-mcp-server — 「MCP コネクタをエージェントに追加する」
- [9-10] https://learn.microsoft.com/ja-jp/azure/sre-agent/custom-mcp-server — 「Builder > Connectors」
- [9-11] https://learn.microsoft.com/ja-jp/azure/sre-agent/connectors — 「Connected / Disconnected / Failed / Initializing / Unavailable」
- [9-12] https://learn.microsoft.com/ja-jp/azure/sre-agent/connectors — 「`{connection-id}/*`」
- [9-13] https://learn.microsoft.com/ja-jp/azure/sre-agent/custom-mcp-server — 「プレイグラウンドでサブエージェントをテストする」

---

10. FAQ から見た GA 後の注意点

説明

FAQ は marketing 的な要約に見えますが、GA 後の導入判断では次の情報が有用です。[10-1]

- 価格には agent compute、conversation/knowledge base の保存、AI model usage、Azure service integration が含まれる。[10-2]
- Azure Monitor logs/metrics 消費や third-party integrations は別料金がかかり得る。[10-2]

- データは AI モデルのトレーニングに使われない。[10-3]

スライド要点

- SRE Agent のコストだけでなく、周辺の Azure Monitor / third-party 側コストを分けて見積もる必要がある。[10-2]
- データ取り扱いの説明は、社内レビュー時に FAQ と privacy/data-privacy への導線を用意しておくと通しやすい。[10-3]

参考（第10章）

- [10-1] https://learn.microsoft.com/ja-jp/azure/sre-agent/faq — 「一般的な FAQ」
- [10-2] https://learn.microsoft.com/ja-jp/azure/sre-agent/faq — 「価格には次のものが含まれます / 別途料金が適用される場合」
- [10-3] https://learn.microsoft.com/ja-jp/azure/sre-agent/faq — 「データは AI モデルのトレーニングに使用されますか? いいえ」

---

11. GA 後に読み替えるべきポイント

1. アクセス制御の説明を更新する

旧稿の「4レイヤー」は、最新 docs（2026-03-28 更新）では「**3レイヤー**」に変更されています。OBO はエージェント権限レイヤーの一部として整理されました。[3-1][3-2]

2. Run modes の既定値を正しく理解する

Response plan・scheduled task の既定は **Autonomous** です。エージェントレベルのグローバル既定は Review ですが、これは個別設定がない場合のフォールバックです。[5-3][5-4]

3. Response plan は subagent routing まで含めて説明する

filters と execution mode だけでなく、どの subagent へルーティングするかが現行 docs の中心です。[7-4][7-5]

4. エージェント権限は Azure RBAC で説明する

managed identity の権限は Reader / Monitoring Reader / Log Analytics Reader などの RBAC 割り当てとして説明する方が、今の docs に合っています。[4-2][4-3]

5. docs 間の差分も注記する

knowledge base のファイル上限のように、ページ間で数字が揺れている箇所は、単に断定せず注記した方が安全です。[8-5][8-8]

6. リファレンス URL の言語を統一する

最新 docs では URL パス構造が変更されたページがあります（例: `roles-permissions-overview` → `user-roles`、`agent-run-modes` → `run-modes`）。リンク切れに注意してください。

---

12. まとめ

GA 後の Azure SRE Agent は、Azure 内部の調査と変更だけでなく、incident platform、scheduled tasks、subagents、connectors、Python tools、MCP を組み合わせて運用ワークフロー全体を自動化する方向へ説明が明確化されました。[1-2][7-1][9-4]

一方で、権限設計はむしろ細かく整理されており、ユーザー ロール、実行モード、エージェント権限（OBO 含む）の 3 レイヤーを分けて考えないと誤読しやすいです。[3-1][4-5][5-1]

資料化するなら、次の 3 枚を先に固めると全体が通しやすいです。

- 「3レイヤーのアクセス制御」
- 「Run mode と OBO の関係」
- 「Incident platform から subagent へ流れる response plan」

---

13. PoC 評価フレームワーク

評価フレームワークの根拠

評価観点は以下の公開フレームワークに基づいて設計しています。

フレームワーク	概要	URL
Microsoft Research AIOps（ICSE 2019）	インシデントライフサイクル KPI（TTD/TTE/TTM）と Insights 分類（Detect/Diagnose/Predict/Optimize）を定義した論文	https://www.microsoft.com/en-us/research/publication/aiops-real-world-challenges-and-research-innovations/
Azure Blog — Advancing Azure service quality with AI: AIOps（2020）	上記と同じ方法論を図解付きで解説。Figure 3: Data→Insights→Actions、Figure 4: TTD/TTE/TTM	https://azure.microsoft.com/en-us/blog/advancing-azure-service-quality-with-artificial-intelligence-aiops/
NIST AI RMF（AI 100-1）	AI リスク管理の汎用フレームワーク。GOVERN/MAP/MEASURE/MANAGE の 4 機能	https://airc.nist.gov/AI_RMF_Interactivity/Playbook

> **注**: AIOps 専用の NIST/ISO 規格は現時点で存在しません。NIST AI RMF は汎用 AI リスク管理フレームワークを AIOps 評価に援用する位置づけです。

評価観点定義

#	評価観点	フレームワーク	フレームワークの意味	定義	SRE Agent での測り方

#	評価観点	フレームワーク	フレームワークの意味	定義	SRE Agent での測り方
1	時間	TTD + TTE + TTM	**T**ime **T**o **D**etect（検知までの時間）+ **T**ime **T**o **E**ngage（担当者着手までの時間）+ **T**ime **T**o **M**itigate（影響緩和までの時間）	検知〜診断〜復旧の合計時間	SRE Agent スレッドのタイムスタンプ差分（インシデント発火 → 診断完了 → 是正完了）
2	品質	Diagnose	MSR Insights 分類の「根因を特定する」。診断が正確かどうかを問う	根因特定と参照情報の正確性	SRE Agent が提示した診断結果・参照ログを運用者がレビューし、見落とし件数をカウント
3	運用負荷	Optimize	MSR Insights 分類の「運用プロセスを最適化する」。人手をどれだけ減らせたかを問う	人手介入がどれだけ減ったか	Review mode での承認回数 / 手動クエリ投入回数を従来フローと比較
4	再現性	GOVERN	NIST AI RMF の「AI の出力を統制・管理する」機能。同じ入力で同じ結果が出るかを問う	同種障害に対する出力の安定性	同一シナリオを 3 回以上再実行し、診断手順と結論の一致率を測定
5	実用性	Actions → Mitigate/Improve	MSR Actions 分類の「今の障害を緩和する（Mitigate）」+「今後の運用を改善する（Improve）」。出力がそのまま使えるかを問う	運用者がそのまま使えるか	PoC 後の運用者アンケート（5 段階）+ 「修正なしで使えた出力」の割合
6	予防性	Predict/Avert	MSR Insights の「障害を予測する（Predict）」+ Actions の「将来の障害を回避する（Avert）」。事後対応ではなく事前に手を打てたかを問う	障害発生前に検知・回避できたか	Scheduled tasks + 自作 ML による事前検知件数
7	可観測性	Data layer	MSR AIOps 方法論の最下層。Insights を出すための元データ（ログ・メトリクス・トレース）が揃っているかを問う	既存監視基盤との接続性	Built-in（Log Analytics/Metrics/ARG）カバー率 + Connectors/Python tools で接続した外部ソース数

シナリオ × SRE Agent 機能マッピング

#	シナリオ	フェーズ	SRE Agent ネイティブで使う機能	自作 ML が補う領域	接続手段
1	障害調査時間短縮	先行	Chat 調査、Log Analytics / Metrics / ARG の built-in クエリ、Response plan による自動トリアージ[9-6][7-4]	—	—
2	原因特定・復旧支援	先行	Memory system（過去インシデント検索[8-1]）、Session insights[8-7]、Subagent routing[7-4]、Knowledge base に runbook 投入[8-5]	類似インシデント分類（過去インシデント群をベクトル化し類似度ランキングを返す軽量モデル）	Python tools[9-7]
3	夜間対応判断	次段階	Response plan の severity filter + Autonomous mode[5-4][7-5]、Scheduled tasks で定期ヘルスチェック[6-1]、Connectors で Teams/PagerDuty 通知[9-4]	アラート相関・重複排除（複数アラートの同時発火パターンを学習し 1 根本原因にまとめるモデル）	Python tools or MCP[9-9]
4	予兆検知	次段階	Scheduled tasks（cron で定期メトリクス監視[6-1]）、Python tools で判定ロジック実行[9-7]	時系列異常検知（Isolation Forest / Prophet 等で閾値超過前に検知するモデル）	Python tools[9-8]

シナリオ別スコアカード

評価観点	#1 障害調査時間短縮	#2 原因特定・復旧支援	#3 夜間対応判断	#4 予兆検知
時間	Chat 調査 vs 手動 KQL の所要時間差	根因到達までの時間（Memory 検索含む）	アラート発火～初動判断の時間	Scheduled task 検知～通知の時間
品質	提示クエリ・診断の正答率	過去インシデント引用の適切性	対応要否判断の正答率	偽陽性率（**自作 ML 精度に依存**）
運用負荷	手動クエリ投入が不要になった割合	runbook 参照の自動化率	夜間の人的エスカレーション削減率	手動閾値監視の代替率
再現性	同種障害での診断手順一致率	Memory + Knowledge base の再現率	Severity filter の一貫性	モデル出力の安定性（**自作 ML**）
実用性	運用者評価（修正なしで使えたか）	復旧手順の実行可能性	夜間当番の負担軽減実感	アラート品質の運用者評価

評価観点	#1 障害調査時間短縮	#2 原因特定・復旧支援	#3 夜間対応判断	#4 予兆検知
予防性	—（reactive シナリオ）	Session insights → 再発防止	繰り返しパターンの Autonomous 化	**事前検知件数（自作 ML 主導）**
可観測性	Built-in カバー率	Connectors 接続の外部ソース数	Incident platform 連携の安定性	Python tools → ML endpoint 接続性

太字 = 自作 ML が主に評価結果に影響するセル

合格ライン（Acceptance Criteria）

評価観点	合格ライン	備考
時間	従来比 50% 短縮	SRE Agent スレッドタイムスタンプで計測
品質	参照漏れ 2 件以下 / シナリオ	運用者によるレビュー判定
運用負荷	手動介入 70% 削減	承認クリック数 + 手動クエリ数の合算
再現性	3 回再実行で 80% 一致	SRE Agent 出力 + 自作 ML 出力の両方を対象
実用性	運用者評価 4/5 以上	アンケート平均
予防性	事前検知 1 件以上 / PoC 期間	#4 のみ。Scheduled tasks + 自作 ML で計測
可観測性	必要データソースの 80% 接続可	Built-in + Connectors + Python tools の合算

自作 ML の位置づけ

手段	どこで使うか	SRE Agent との接続	PoC での扱い
類似インシデント分類	#2 原因特定	Python tools → embedding API 呼び出し	先行フェーズ後半で試行
アラート相関モデル	#3 夜間対応	Python tools or MCP server	次段階で設計
時系列異常検知	#4 予兆検知	Python tools → Azure ML endpoint	次段階で設計

先行シナリオ（#1, #2）は SRE Agent ネイティブ機能だけでほぼ完結します。自作 ML は #2 後半〜#3, #4 の次段階で段階的に組み込む形にすると、PoC の複雑性を抑えつつ ML の効果も検証できます。